

The RAG Pipeline, End to End

From raw documents to a structured, grounded answer — every stage, the **why** behind it, and exactly which LangChain component does the job.

Format: Problem → intuition → component → code

Stack: LangChain · OpenAI / HuggingFace · FAISS / Chroma

Why this document exists. Most learners can recite the words "load, chunk, embed, retrieve, generate" but freeze when asked *where* the overlap goes, *why* we re-rank, or *which* splitter to pick. This module walks the pipeline as two clear phases and pins a concrete tool to every box.

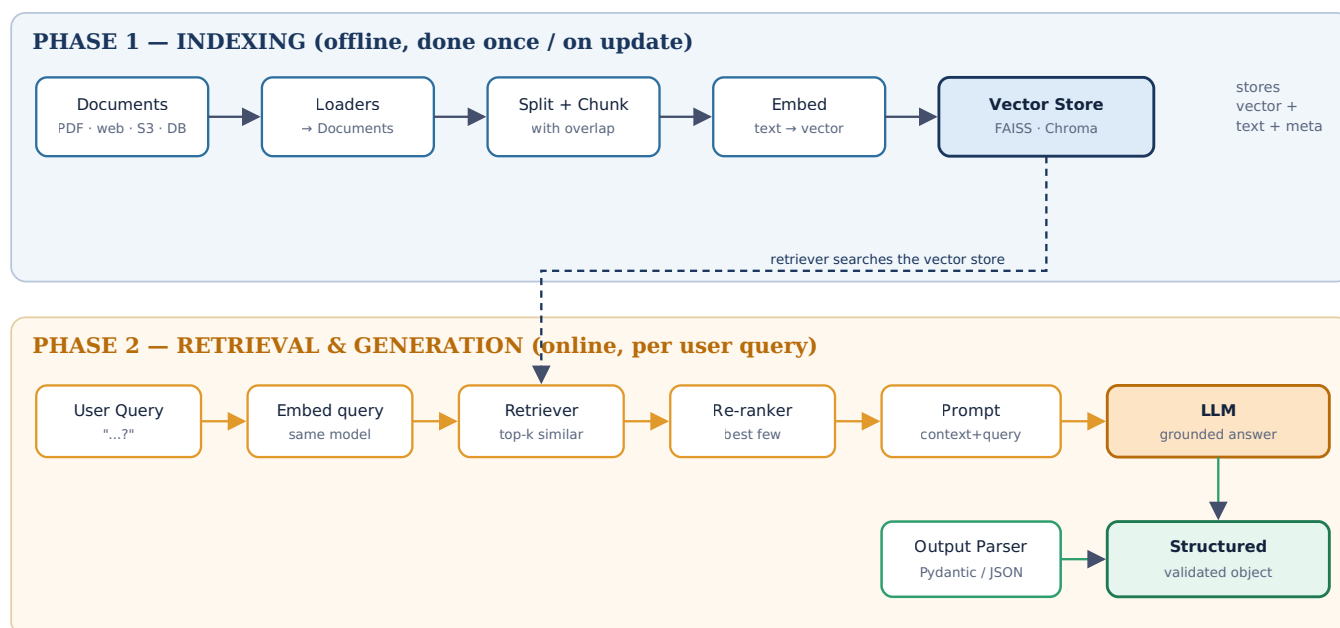


Fig 1 — The whole pipeline at a glance. Blue = built once and reused. Orange = runs on every question. Green = shaping the answer.

The one-line mental model: Indexing turns your knowledge into a searchable *vector library*. At query time we fetch only the few most relevant pages, hand them to the LLM as evidence, and force the answer into a clean structure. The LLM never "memorises" your data — it *reads* the retrieved snippets each time.

Phase 1 — Indexing the Knowledge

OFFLINE

1 Document Loading

Knowledge lives in many places and formats. A **loader's** only job is to read a source and return a list of LangChain `Document` objects — each holding `page_content` (the text) and `metadata` (source, page number, URL, etc.). Metadata matters: it is what lets you later show citations.

Source	LangChain loader	Underlying library
Local PDF	PyPDFLoader, PyMuPDFLoader	pypdf / PyMuPDF
Remote web page	WebBaseLoader	requests + BeautifulSoup
Amazon S3 (one file)	S3FileLoader	boto3 + unstructured
Amazon S3 (whole bucket/prefix)	S3DirectoryLoader	boto3 + unstructured
Folder of mixed files	DirectoryLoader	unstructured
CSV / tabular	CSVLoader	csv
Notion / Confluence / DB	NotionDBLoader, SQLDatabaseLoader	connector SDKs

```
# Load from three different sources – all return List[Document]
from langchain_community.document_loaders import PyPDFLoader, WebBaseLoader, S3DirectoryLoader

pdf_docs = PyPDFLoader("handbook.pdf").load()
web_docs = WebBaseLoader("https://docs.mycompany.com/policy").load()
s3_docs = S3DirectoryLoader(bucket="kb-prod", prefix="manuals/").load()

docs = pdf_docs + web_docs + s3_docs
print(docs[0].page_content[:120]) # the text
print(docs[0].metadata)           # {'source': 'handbook.pdf', 'page': 0}
```

Intuition: Don't think of a loader as "reading a file." Think of it as a *normaliser* — no matter how weird the source, everything downstream only ever sees the same tidy `Document` shape.

2 Splitting & Chunking (with Overlap)

A loaded document can be thousands of words. We cannot feed that whole thing into an embedding model or a retriever — so we cut it into smaller **chunks**.

Why split at all? Three hard reasons:

- **Embedding limit** — embedding models accept a bounded number of tokens; a 50-page doc simply won't fit in one vector.
- **Retrieval precision** — if a chunk is a whole chapter, its single vector becomes a blurry "average" of many topics, so similarity search gets vague. Small focused chunks → sharp matches.
- **Context window & cost** — at answer time we only stuff the *few* needed chunks into the prompt instead of the entire corpus.

Two key knobs: `chunk_size` (max characters/tokens per chunk) and `chunk_overlap` (how many characters the *end* of one chunk repeats at the *start* of the next).

Why overlap? A naive cut can slice a sentence — or an idea — right down the middle. The first half lands in chunk A, the second half in chunk B, and *neither* chunk is now individually answerable. Overlap creates a sliding "safety margin" so context that straddles a boundary survives in at least one chunk.

Source text (chunk_size = 100 chars, chunk_overlap = 20 chars)

...the transformer uses self-attention so every token can look at every other token in the sequence at once...

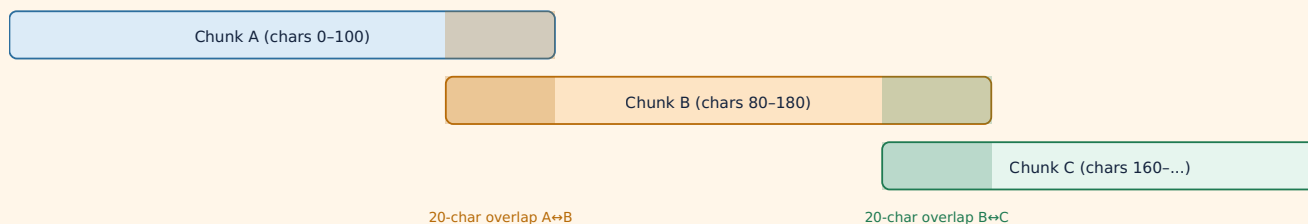


Fig 2 — Each chunk repeats the last 20 characters of the previous one. The shaded strips are the shared text that guarantees no idea is orphaned at a boundary.

Worked example. Take the sentence and chop it at `chunk_size=60, chunk_overlap=15` :

```
# text = "Self-attention lets every word attend to every other word." (58 chars → 1 chunk)
# Longer text, say 130 chars, becomes:
Chunk 0 : "Self-attention lets the model weigh how important each ot"
Chunk 1 : "rtant each other token is when encoding the current token."
          |_____ 15 chars repeated from end of Chunk 0 _____|
```

Which splitter? LangChain ships several. The default you should reach for is the **RecursiveCharacterTextSplitter**.

Splitter	How it cuts	Best for
<code>RecursiveCharacterTextSplitter</code>	Tries separators in order: paragraph → line → sentence → word, so it cuts at the most natural boundary that still fits the size.	Default choice — general prose, PDFs, docs.
<code>CharacterTextSplitter</code>	Splits on a single fixed separator (e.g. <code>"\n\n"</code>).	Simple, uniform text.
<code>TokenTextSplitter</code>	Counts <i>tokens</i> (not chars) using the model's tokenizer.	When you must respect an exact token budget.
<code>MarkdownHeaderTextSplitter</code>	Splits by <code>#</code> / <code>##</code> headers and keeps the header in metadata.	Markdown / structured docs.
<code>SemanticChunker</code>	Cuts where the <i>meaning</i> shifts (uses embeddings to detect topic change).	When topic-clean chunks matter most.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,      # ~1000 chars per chunk
    chunk_overlap=150,    # 150 chars bleed into the next chunk
    separators=["\n\n", "\n", ". ", " ", ""], # tried in this order
)
chunks = splitter.split_documents(docs)
print(f"{len(docs)} docs → {len(chunks)} chunks")
```

Rule of thumb: overlap $\approx$ 10–20% of `chunk_size` . Too little → broken ideas; too much → duplicate, redundant retrievals and a bigger index.

3 Embedding — Turning Text into Vectors

An **embedding** is a list of numbers (a vector, e.g. 384 or 1536 dimensions) that captures the *meaning* of a chunk. Texts with similar meaning land close together in this vector space — that closeness is what search will exploit later.

Provider	LangChain class	Notes
OpenAI	<code>OpenAIEmbeddings</code>	<code>text-embedding-3-small</code> , hosted, paid, high quality.
HuggingFace (local)	<code>HuggingFaceEmbeddings</code>	<code>all-MiniLM-L6-v2</code> , <code>BGE</code> — free, runs offline.
Cohere	<code>CohereEmbeddings</code>	Strong multilingual support.

```
from langchain_openai import OpenAIEmbeddings
embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vec = embeddings.embed_query("What is self-attention?") # → [0.013, -0.072, ...] (1536 floats)
```

Critical rule: the *same* embedding model must be used for chunks (now) and for the user's query (later). Two different models produce two different, incomparable spaces — like measuring one side in metres and the other in feet.

4 Storing in a Vector Store

The **vector store** (vector database) keeps each chunk's vector alongside its original text and metadata, and provides fast *nearest-neighbour* search over millions of vectors.

Vector store	Type	When to use
<code>FAISS</code>	Local, in-memory/file	Prototyping, single machine, fast.
<code>Chroma</code>	Local, persistent	Easy local dev with persistence.
<code>Pinecone</code> / <code>Qdrant</code> / <code>Weaviate</code> / <code>Milvus</code>	Managed / server	Production scale, filtering, HA.

```
from langchain_community.vectorstores import FAISS

vectorstore = FAISS.from_documents(chunks, embeddings) # embeds + indexes in one call
vectorstore.save_local("faiss_index")                # persist to disk
```

Indexing is now complete. Everything above runs *once* (or whenever documents change). The result — a populated vector store — is the only artefact Phase 2 needs.

Phase 2 — Retrieval & Generation

PER QUERY

5 Query → Query Embedding

When a user asks a question, we convert that question into a vector using the *exact same* embedding model from Step 3. Now the query and the chunks live in one comparable space.

```
query = "How does self-attention handle long sentences?"
q_vec = embeddings.embed_query(query) # same model as the chunks
```

6 Retriever — Fetch the Top-k Chunks

A **retriever** wraps the vector store and answers one question: "which stored chunks are closest to this query vector?" It returns the top-**k** (say 4) most similar chunks. Closeness is measured by cosine similarity / distance.

```
retriever = vectorstore.as_retriever(
    search_type="similarity",      # or "mmr" for diverse results
    search_kwargs={"k": 4},       # return 4 chunks
)
top_chunks = retriever.invoke(query)  # List[Document], most relevant first
```

Search types: **similarity** just grabs the nearest neighbours. **mmr** (Maximal Marginal Relevance) grabs neighbours that are relevant *and* different from each other — useful to avoid 4 near-duplicate chunks.

7 Re-ranking — Why a Second Pass?

The retriever is **fast but approximate**. It compares a single query vector against single chunk vectors (a "bi-encoder"). That speed comes at a cost: nuance gets lost, so the true best answer might sit at rank 7, not rank 1.

Why re-rank? A **cross-encoder** re-ranker reads the query and a chunk *together* in one pass and scores how well that specific chunk actually answers *that specific query*. It is far more accurate — but too slow to run over the whole database. So we use the best of both: retriever fetches a wide net (e.g. top 25), the re-ranker carefully re-scores just those 25 and keeps the top 3-4. This is especially valuable for **complex or multi-part queries** where surface similarity is misleading.

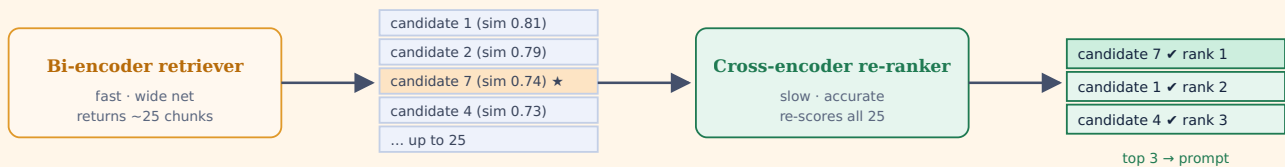


Fig 3 — The hidden gem (candidate 7) was only 3rd by raw similarity but is actually the best answer. The cross-encoder promotes it to rank 1.

```
# Wrap the retriever with a re-ranker (ContextualCompressionRetriever pattern)
from langchain.retrievers import ContextualCompressionRetriever
from langchain_cohere import CohereRerank  # or a local BGE / cross-encoder reranker

reranker = CohereRerank(model="rerank-english-v3.0", top_n=3)
compression_retriever = ContextualCompressionRetriever(
    base_retriever=vectorstore.as_retriever(search_kwargs={"k": 25}),
    base_compressor=reranker,
)
final_chunks = compression_retriever.invoke(query)  # best 3, carefully scored
```

8 Building the Prompt (Context + Query)

Now we assemble the **prompt**: the retrieved chunks become the "context", the user's question is appended, and a system instruction tells the LLM to answer *only* from that context. This grounding is what stops hallucination.

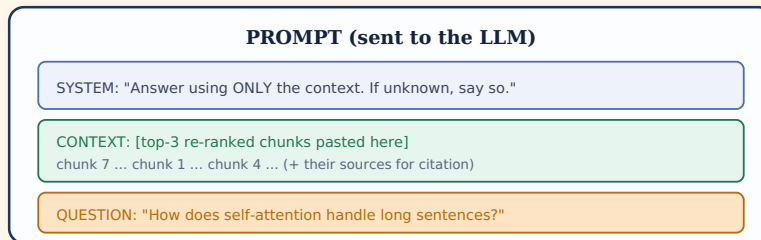


Fig 4 — The prompt is just three stacked parts: instructions, retrieved evidence, and the user's question.

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_template("""
You are a helpful assistant. Answer using ONLY the context below.
If the answer is not in the context, say you don't know.

Context:
{context}

Question: {question}
""")
```

9 The LLM Generates an Answer

The assembled prompt goes to the LLM (e.g. `ChatOpenAI`). Because the evidence is right there in the prompt, the answer is *grounded* in your documents rather than the model's memory.

```
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
```

10 Output Parser — Forcing Clean Structure

Raw LLM output is free-form text. Real applications need a predictable shape (JSON, a typed object) so downstream code can use it. A **PydanticOutputParser** solves this in two moves: it (a) injects *format instructions* into the prompt telling the model exactly what JSON to emit, and (b) parses + validates the model's string back into a typed Python object.

Parser	What it returns
<code>PydanticOutputParser</code>	A validated Pydantic model instance (type-checked fields).
<code>JsonOutputParser</code>	A plain Python <code>dict</code> from JSON.
<code>StructuredOutputParser</code>	A dict from a lightweight schema (no Pydantic).
<code>StrOutputParser</code>	Just the raw string (when no structure is needed).

```
from langchain_core.output_parsers import PydanticOutputParser
from pydantic import BaseModel, Field

class Answer(BaseModel):
    answer: str = Field(description="the direct answer")
    confidence: float = Field(description="0-1 confidence")
    sources: list[str] = Field(description="source files used")

parser = PydanticOutputParser(pydantic_object=Answer)
# parser.get_format_instructions() → text injected into the prompt
# LLM returns a JSON string → parser turns it into a typed Answer object:
```

```
# Answer(answer='Self-attention compares every token...', confidence=0.92,
sources=['handbook.pdf'])
```

Intuition: the parser is a two-way handshake. Going *out*, it whispers to the LLM "please reply in exactly this JSON." Coming *back*, it refuses anything that doesn't fit and hands your code a clean, validated object.

Putting It All Together (LCEL Chain)

LangChain Expression Language (LCEL) lets you wire the runtime pieces with the `|` pipe operator, exactly mirroring Fig 1's orange path:

```
from langchain_core.runnables import RunnablePassthrough

def format_docs(docs):
    return "\n\n".join(d.page_content for d in docs)

rag_chain = (
    {"context": compression_retriever | format_docs, # retrieve + rerank + flatten
     "question": RunnablePassthrough()}           # pass query straight through
    | prompt                                       # Step 8: build prompt
    | llm                                         # Step 9: generate
    | parser                                     # Step 10: structure it
)

result = rag_chain.invoke("How does self-attention handle long sentences?")
print(result.answer, result.confidence, result.sources)
```

One-Page Component Cheat Sheet

#	Stage	What it does	LangChain / library component
1	Load	Read any source → Document objects	PyPDFLoader, WebBaseLoader, S3DirectoryLoader
2	Split / Chunk	Cut into overlapping chunks	RecursiveCharacterTextSplitter
3	Embed	Text → meaning vector	OpenAIEmbeddings, HuggingFaceEmbeddings
4	Store	Index vectors for fast search	FAISS, Chroma, Pinecone
5	Embed query	Question → vector (same model)	embeddings.embed_query()
6	Retrieve	Fetch top-k similar chunks	vectorstore.as_retriever()
7	Re-rank	Re-score candidates accurately	ContextualCompressionRetriever + CohereRerank
8	Prompt	Context + question → prompt	ChatPromptTemplate
9	Generate	LLM produces grounded answer	ChatOpenAI
10	Parse	Text → validated structure	PydanticOutputParser

Build a searchable vector library once (load → chunk → embed → store). For every question, turn it into a vector, pull the closest chunks, re-rank to surface the truly best ones, paste them into a prompt as evidence, let the LLM answer from that evidence, and force the reply into clean JSON.